

OpenMP Exam Prep

30 Questions — Using Only Miss's 5 Directives

Key Directives (from slides)

Directive	Purpose
<code>#pragma omp parallel</code>	Creates a team of threads that run the block together.
<code>#pragma omp parallel for</code>	Splits a for-loop's iterations across threads.
<code>private(var)</code>	Each thread gets its own private copy of a variable.
<code>reduction(+:sum)</code>	Safely combines a variable (e.g. sum) across all threads. Also works with *, max, min in place of +.
<code>omp_get_thread_num()</code>	Returns the calling thread's ID number.

The Golden Rule

Note: Every question below uses ONLY these 5 directives. The teacher may change the logic (sum becomes factorial, becomes product, becomes even/odd count, becomes positive/negative count...) but the skeleton never changes: `#pragma omp parallel for` splits the loop, and `reduction(op:var)` safely combines the result. Master ONE pattern and you can answer almost all of them. A few questions (Q1, Q2, Q22, Q23, Q25, Q26, Q29, Q30) use plain `#pragma omp parallel` or no reduction at all — these are marked clearly.

What's Inside — All 30 Questions

1. Write a simple OpenMP program that prints a message from multiple threads.
2. Modify the program so each thread prints its own thread ID.
3. Parallelize a simple for loop so its iterations run on different threads.
4. Write a program to calculate the sum of all elements of an array using OpenMP.
5. Modify the sum program to instead calculate the PRODUCT of all array elements.
6. Write an OpenMP program to calculate the factorial of a number N.
7. Find the MAXIMUM value in an array using OpenMP.
8. Find the MINIMUM value in an array using OpenMP.
9. Count how many even numbers and how many odd numbers are in an array.
10. Calculate the sum of squares of numbers from 1 to N using OpenMP.
11. Calculate the sum of only the ODD numbers in an array using OpenMP.
12. Count how many elements in an array are greater than a given value.
13. Calculate the average of all elements in an array using OpenMP.
14. Calculate the sum of cubes of numbers from 1 to N using OpenMP.
15. Calculate the sum of the first N natural numbers using OpenMP.
16. Calculate the sum of only the EVEN numbers in an array using OpenMP.
17. Calculate the product of only the EVEN numbers in an array using OpenMP.
18. Count how many NEGATIVE numbers are in an array using OpenMP.
19. Count how many elements in an array are LESS THAN a given value.
20. Find both the MAXIMUM and MINIMUM value of an array in the same loop.
21. Calculate the sum of squares of only the EVEN numbers in an array.
22. What is the difference between a PRIVATE and a SHARED variable? Show what goes wrong without private.
23. Print which thread computed which loop iteration.

24. Calculate the sum AND the count of array elements together, then find the average.
25. Print the multiplication table of a number using OpenMP (parallel for, no reduction).
26. Find the highest thread ID number used while printing from all threads.
27. Calculate the sum of the ABSOLUTE VALUES of array elements (array may contain negatives).
28. Count how many elements are POSITIVE and how many are NEGATIVE in the same array.
29. Why is reduction needed? Show a manual (unsafe) sum using `#pragma omp parallel` to explain.
30. What is the difference between `#pragma omp parallel` and `#pragma omp parallel for`?

Q1. Write a simple OpenMP program that prints a message from multiple threads.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>
int main() {
    #pragma omp parallel
    {
        printf("Hello from thread\n");
    }
    return 0;
}
```

Sample Output

```
Hello from thread
Hello from thread
Hello from thread
Hello from thread
```

Explanation — Step by Step (Simple Words)

1. Include <omp.h> — this gives you all OpenMP functions and directives.
2. #pragma omp parallel tells the compiler: 'run the block right after this on multiple threads at the same time', creating a team of threads.
3. The { } block is the parallel region — every thread runs its own copy of everything inside it.
4. If the system has 4 cores, OpenMP typically creates 4 threads, so printf runs 4 times.
5. The order the lines print in is NOT guaranteed — threads can finish in any order.

Note: This is the base template every other question builds on.

Q2. Modify the program so each thread prints its own thread ID.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>
int main() {
    int id;
    #pragma omp parallel private(id)
    {
        id = omp_get_thread_num();
        printf("Hello from thread %d\n", id);
    }
    return 0;
}
```

Sample Output

```
Hello from thread 0
Hello from thread 1
Hello from thread 2
Hello from thread 3
```

Explanation — Step by Step (Simple Words)

1. `omp_get_thread_num()` returns the ID of the thread currently running it (0, 1, 2, 3, ...).
2. `private(id)` gives each thread its OWN separate copy of `id`, so threads never overwrite each other's value.
3. Without `private(id)`, all threads would share ONE `id` variable and the printed numbers could get mixed up.
4. Thread 0 is always the original/master thread; 1, 2, 3... are the extra worker threads OpenMP created.

Note: Any time a variable must hold a DIFFERENT value per thread, make it private.

Q3. Parallelize a simple for loop so its iterations run on different threads.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>
int main() {
    #pragma omp parallel for
    for (int i = 0; i < 10; i++) {
        printf("i = %d\n", i);
    }
    return 0;
}
```

Sample Output

```
i = 0
i = 4
i = 1
i = 5
i = 2
...
(exact order may vary each run)
```

Explanation — Step by Step (Simple Words)

1. `#pragma omp parallel for` does two things at once: creates a team of threads, and automatically splits the loop's iterations among them.
2. Example: with 10 iterations and 4 threads, roughly thread 0 gets `i=0,1,2`; thread 1 gets `i=3,4,5`; and so on — OpenMP decides the exact split.
3. The loop variable `i` is automatically made private for you — you do not need to write `private(i)` yourself for the loop counter.
4. Because iterations run at the same time on different cores, the printed order is scrambled — this is expected.

Note: This is the base 'split the loop' skeleton — Q4 onward just adds reduction on top of it.

Q4. Write a program to calculate the sum of all elements of an array using OpenMP.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>
int main() {
    int i, sum = 0;
    int arr[10] = {1,2,3,4,5,6,7,8,9,10};
    #pragma omp parallel for reduction(+:sum)
    for (i = 0; i < 10; i++) {
        sum = sum + arr[i];
    }
    printf("Total Sum = %d\n", sum);
    return 0;
}
```

Sample Output

Total Sum = 55

Explanation — Step by Step (Simple Words)

1. The array holds 1 to 10. Expected sum = $1+2+\dots+10 = 55$, so you can double-check easily.
2. `reduction(+:sum)` is the key part: each thread keeps its OWN private mini-total of sum, adds up its own share of the array, and at the end OpenMP ADDS all the mini-totals together into the final sum.
3. Why not just make sum shared? Because if 4 threads update the same shared sum at the exact same time, some additions get LOST — this is a race condition. reduction avoids this automatically.
4. The `+` in `reduction(+:sum)` is the operator being reduced — this same slot can also hold `*`, `max`, or `min` (see the next questions).
5. After the loop, sum already holds the fully combined, correct total.

Note: This is Miss's exact slide example — memorize this pattern first, everything else is a small variation of it.

Q5. Modify the sum program to instead calculate the PRODUCT of all array elements.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>
int main() {
    int i;
    long product = 1;
    int arr[5] = {1,2,3,4,5};
```

```
#pragma omp parallel for reduction(*:product)
for (i = 0; i < 5; i++) {
    product = product * arr[i];
}
printf("Total Product = %ld\n", product);
return 0;
}
```

Sample Output

Total Product = 120

Explanation — Step by Step (Simple Words)

1. Only two things changed from the sum example: reduction operator is now * instead of +, and the loop body multiplies instead of adds.
2. product starts at 1 (not 0!) — multiplying by 0 would wipe everything out. Sums start at 0, products start at 1. This is the 'identity value' rule.
3. Each thread multiplies its own chunk of the array into its private copy of product, then OpenMP multiplies all the private copies together at the end.
4. $1 \times 2 \times 3 \times 4 \times 5 = 120$, matching the printed result.

Note: Proves the golden rule: same skeleton as Q4, only the operator and starting value changed.

Q6. Write an OpenMP program to calculate the factorial of a number N.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>
int main() {
    int n = 6;
    long fact = 1;
    #pragma omp parallel for reduction(*:fact)
    for (int i = 1; i <= n; i++) {
        fact = fact * i;
    }
    printf("Factorial of %d = %ld\n", n, fact);
    return 0;
}
```

Sample Output

Factorial of 6 = 720

Explanation — Step by Step (Simple Words)

1. Factorial of N means $N \times (N-1) \times \dots \times 1$. Here $N=6$, so $6 \times 5 \times 4 \times 3 \times 2 \times 1 = 720$.
2. This is the SAME pattern as Q5 (product reduction) — just looping from 1 to n instead of looping over array elements.

3. `reduction(*:fact)` again gives every thread a private safe copy of `fact` so multiplications from different threads don't clash.
4. `fact` starts at 1, same identity-value rule as Q5.

Note: If the teacher asks for factorial instead of sum, this is exactly the swap: `+` becomes `*`, array loop becomes `1..n` loop.

Q7. Find the MAXIMUM value in an array using OpenMP.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>
int main() {
    int arr[8] = {12, 45, 3, 67, 21, 9, 88, 34};
    int max_val = arr[0];
    #pragma omp parallel for reduction(max:max_val)
    for (int i = 0; i < 8; i++) {
        if (arr[i] > max_val) {
            max_val = arr[i];
        }
    }
    printf("Maximum value = %d\n", max_val);
    return 0;
}
```

Sample Output

Maximum value = 88

Explanation — Step by Step (Simple Words)

1. `max_val` starts at `arr[0]` so it already holds a valid value from the array.
2. `reduction(max:max_val)` gives each thread its own private 'best value found so far' from its own chunk of the array.
3. Inside the loop, each thread just checks: 'is this element bigger than my current best? If yes, update it.'
4. At the end, OpenMP compares all threads' private best values and keeps the overall largest one.

Note: Works the same way for `reduction(min:...)` in Q8 — just flip `'>'` to `'<'`.

Q8. Find the MINIMUM value in an array using OpenMP.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>
int main() {
```

```

int arr[8] = {12, 45, 3, 67, 21, 9, 88, 34};
int min_val = arr[0];
#pragma omp parallel for reduction(min:min_val)
for (int i = 0; i < 8; i++) {
    if (arr[i] < min_val) {
        min_val = arr[i];
    }
}
printf("Minimum value = %d\n", min_val);
return 0;
}

```

Sample Output

Minimum value = 3

Explanation — Step by Step (Simple Words)

1. Mirror image of Q7 — everything identical except reduction changes from max to min, and the comparison flips from '>' to '<'.
2. min_val starts at arr[0] as a safe starting guess.
3. Each thread privately keeps its own smallest-value-found-so-far.
4. OpenMP combines all threads' private smallest values and keeps the overall smallest as the answer.

Note: Q7 and Q8 prove it again: memorize ONE pattern, swap one word to answer both.

Q9. Count how many even numbers and how many odd numbers are in an array.

Answer (Code)

```

#include <stdio.h>
#include <omp.h>
int main() {
    int arr[10] = {1,2,3,4,5,6,7,8,9,10};
    int evenCount = 0, oddCount = 0;
    #pragma omp parallel for reduction(+:evenCount, oddCount)
    for (int i = 0; i < 10; i++) {
        if (arr[i] % 2 == 0)
            evenCount++;
        else
            oddCount++;
    }
    printf("Even count = %d\n", evenCount);
    printf("Odd count = %d\n", oddCount);
    return 0;
}

```

Sample Output

Even count = 5

Odd count = 5

Explanation — Step by Step (Simple Words)

1. You can reduce MORE THAN ONE variable at once — just separate them with a comma inside the same reduction(...) clause.
2. Each thread keeps its own private evenCount and oddCount, increasing whichever one matches the number it's checking ($\text{arr}[i] \% 2 == 0$ means even).
3. At the end, OpenMP adds together all threads' private evenCount values into one final evenCount, and the same for oddCount.
4. Out of 1 to 10 there are exactly 5 even (2,4,6,8,10) and 5 odd (1,3,5,7,9) numbers.

Note: Common exam twist: 'count elements above a value', 'count negatives' — same reduction(+:count) idea, see Q12 and Q18.

Q10. Calculate the sum of squares of numbers from 1 to N using OpenMP.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>
int main() {
    int n = 5;
    long sumSquares = 0;
    #pragma omp parallel for reduction(+:sumSquares)
    for (int i = 1; i <= n; i++) {
        sumSquares += i * i;
    }
    printf("Sum of squares 1..%d = %ld\n", n, sumSquares);
    return 0;
}
```

Sample Output

Sum of squares 1..5 = 55

Explanation — Step by Step (Simple Words)

1. $1^2+2^2+3^2+4^2+5^2 = 1+4+9+16+25 = 55$ — matches the printed result.
2. Same sum-reduction skeleton as Q4, just with $i*i$ inside the loop instead of $\text{arr}[i]$, and looping directly over numbers 1..n instead of reading from an array.
3. `reduction(+:sumSquares)` keeps a safe private running total per thread, then adds them all together at the end.
4. Confirms the golden rule again: same pragma, only the math inside the loop body changes.

Note: Any 'sum of ... from 1 to N' question (cubes, odd numbers, etc.) follows this identical pattern.

Q11. Calculate the sum of only the ODD numbers in an array using OpenMP.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>
int main() {
    int i, oddSum = 0;
    int arr[10] = {1,2,3,4,5,6,7,8,9,10};
    #pragma omp parallel for reduction(+:oddSum)
    for (i = 0; i < 10; i++) {
        if (arr[i] % 2 != 0) {
            oddSum = oddSum + arr[i];
        }
    }
    printf("Sum of odd numbers = %d\n", oddSum);
    return 0;
}
```

Sample Output

Sum of odd numbers = 25

Explanation — Step by Step (Simple Words)

1. $1+3+5+7+9 = 25$ — only the odd-valued numbers are added, not all ten numbers.
2. This is Q4's sum pattern with one small addition: an if-check inside the loop before adding to the reduction variable.
3. Each thread still privately totals oddSum for its own chunk of the array, only adding when $\text{arr}[i] \% 2 \neq 0$ (meaning the number is odd).
4. OpenMP combines all the private oddSum totals at the end into the final answer, exactly like the plain sum example.

Note: Shows that reduction still works perfectly even when the loop body has an if-condition inside it.

Q12. Count how many elements in an array are greater than a given value.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>
int main() {
    int i, count = 0;
    int arr[10] = {5, 22, 9, 41, 3, 60, 18, 7, 35, 12};
    int threshold = 20;
    #pragma omp parallel for reduction(+:count)
    for (i = 0; i < 10; i++) {
        if (arr[i] > threshold) {
            count++;
        }
    }
}
```

```

}
printf("Numbers greater than %d = %d\n", threshold, count);
return 0;
}

```

Sample Output

Numbers greater than 20 = 4

Explanation — Step by Step (Simple Words)

1. The values above 20 are 22, 41, 60, 35 — that's 4 numbers, matching the printed result.
2. count starts at 0 (identity value for a + reduction, same rule as every sum example).
3. Each thread privately increments its own copy of count only when its element passes the condition (`arr[i] > threshold`).
4. `reduction(+:count)` then adds all the private counts together to give the final total.

Note: Exactly the same shape as Q9's even/odd counting, just with one condition instead of two.

Q13. Calculate the average of all elements in an array using OpenMP.

Answer (Code)

```

#include <stdio.h>
#include <omp.h>
int main() {
    int i, sum = 0;
    int arr[10] = {1,2,3,4,5,6,7,8,9,10};
    int n = 10;
    float average;
    #pragma omp parallel for reduction(+:sum)
    for (i = 0; i < n; i++) {
        sum = sum + arr[i];
    }
    average = (float)sum / n;
    printf("Sum = %d, Average = %.2f\n", sum, average);
    return 0;
}

```

Sample Output

Sum = 55, Average = 5.50

Explanation — Step by Step (Simple Words)

1. This is Q4's exact sum program with ONE extra line added after the parallel region: divide the final sum by n to get the average.
2. Notice the division happens OUTSIDE the parallel region, on the main/master thread only, after all threads have already finished combining sum — it does not need its own pragma.

3. (float)sum forces the division to produce a decimal answer (5.50) instead of doing whole-number integer division (which would incorrectly give 5).
4. $55 / 10 = 5.50$, matching the printed result.

Note: A good reminder: reduction only needs to wrap the part done by many threads — simple follow-up math after the loop stays normal, single-threaded code.

Q14. Calculate the sum of cubes of numbers from 1 to N using OpenMP.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>
int main() {
    int n = 4;
    long sumCubes = 0;
    #pragma omp parallel for reduction(+:sumCubes)
    for (int i = 1; i <= n; i++) {
        sumCubes += i * i * i;
    }
    printf("Sum of cubes 1..%d = %ld\n", n, sumCubes);
    return 0;
}
```

Sample Output

Sum of cubes 1..4 = 100

Explanation — Step by Step (Simple Words)

1. $1^3 + 2^3 + 3^3 + 4^3 = 1 + 8 + 27 + 64 = 100$ — matches the printed result.
2. Identical skeleton to Q10 (sum of squares) — only $i*i*i$ replaces $i*i$ inside the loop.
3. `reduction(+:sumCubes)` keeps a private running total per thread, then OpenMP adds them all together at the end.
4. This is the pattern to remember for ANY 'sum of powers from 1 to N' style question.

Note: Last proof of the golden rule from Miss's original set: 14 different-looking questions, one single reusable skeleton.

Q15. Calculate the sum of the first N natural numbers using OpenMP.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>
int main() {
    int n = 10;
```

```

long sum = 0;
#pragma omp parallel for reduction(+:sum)
for (int i = 1; i <= n; i++) {
    sum += i;
}
printf("Sum of first %d natural numbers = %ld\n", n, sum);
return 0;
}

```

Sample Output

Sum of first 10 natural numbers = 55

Explanation — Step by Step (Simple Words)

1. 'First N natural numbers' just means 1, 2, 3, ... N. Here N=10, so $1+2+\dots+10 = 55$.
2. Same skeleton as Q10/Q14 — loop directly over 1..n instead of reading an array, and add i itself (not $i*i$ or $i*i*i$).
3. `reduction(+:sum)` gives each thread a private running total for its own share of the numbers.
4. OpenMP adds all the private totals together at the end to get the final sum.

Note: If the teacher just says 'sum of numbers from 1 to N' with no squaring or cubing, this plain version is the answer.

Q16. Calculate the sum of only the EVEN numbers in an array using OpenMP.

Answer (Code)

```

#include <stdio.h>
#include <omp.h>
int main() {
    int i, evenSum = 0;
    int arr[10] = {1,2,3,4,5,6,7,8,9,10};
    #pragma omp parallel for reduction(+:evenSum)
    for (i = 0; i < 10; i++) {
        if (arr[i] % 2 == 0) {
            evenSum = evenSum + arr[i];
        }
    }
    printf("Sum of even numbers = %d\n", evenSum);
    return 0;
}

```

Sample Output

Sum of even numbers = 30

Explanation — Step by Step (Simple Words)

1. $2+4+6+8+10 = 30$ — only the even-valued numbers are added.
2. Mirror image of Q11 (sum of odd numbers) — the condition just flips from $\% 2 \neq 0$ to $\% 2 == 0$.

3. Each thread privately totals evenSum for its own chunk, only adding when the element is even.
4. reduction(+:evenSum) safely combines every thread's private total into the final answer.

Note: Pairs with Q11 — together they prove the if-condition trick works for either odd or even filtering.

Q17. Calculate the product of only the EVEN numbers in an array using OpenMP.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>
int main() {
    int i;
    long evenProduct = 1;
    int arr[6] = {2,3,4,5,6,7};
    #pragma omp parallel for reduction(*:evenProduct)
    for (i = 0; i < 6; i++) {
        if (arr[i] % 2 == 0) {
            evenProduct = evenProduct * arr[i];
        }
    }
    printf("Product of even numbers = %ld\n", evenProduct);
    return 0;
}
```

Sample Output

Product of even numbers = 48

Explanation — Step by Step (Simple Words)

1. The even numbers in the array are 2, 4, 6. Their product is $2 \times 4 \times 6 = 48$.
2. Combines two earlier tricks at once: the * reduction from Q5, and the if-condition filter from Q11/Q16.
3. evenProduct starts at 1 (identity value for multiplication), never 0.
4. Each thread multiplies only the even elements in its own chunk into its private evenProduct, then OpenMP multiplies all the private copies together.

Note: Whenever a question says 'only even/odd/positive elements', just add one if-check inside the loop — the pragma line never changes.

Q18. Count how many NEGATIVE numbers are in an array using OpenMP.

Answer (Code)

```
#include <stdio.h>
```

```
#include <omp.h>
int main() {
    int i, negCount = 0;
    int arr[8] = {-5, 3, -2, 8, -9, 4, -1, 7};
    #pragma omp parallel for reduction(+:negCount)
    for (i = 0; i < 8; i++) {
        if (arr[i] < 0) {
            negCount++;
        }
    }
    printf("Negative count = %d\n", negCount);
    return 0;
}
```

Sample Output

Negative count = 4

Explanation — Step by Step (Simple Words)

1. The negative values are -5, -2, -9, -1 — that is 4 numbers, matching the printed result.
2. Exactly the same shape as Q12 (count greater than threshold) — the condition just checks `arr[i] < 0` instead of `arr[i] > threshold`.
3. `negCount` starts at 0, the identity value for a + reduction.
4. Each thread privately counts negatives in its own chunk; `reduction(+:negCount)` adds all the private counts together.

Note: 'Count how many are negative/positive/zero' is always this same counting skeleton with a different if-condition.

Q19. Count how many elements in an array are LESS THAN a given value.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>
int main() {
    int i, count = 0;
    int arr[10] = {5, 22, 9, 41, 3, 60, 18, 7, 35, 12};
    int threshold = 20;
    #pragma omp parallel for reduction(+:count)
    for (i = 0; i < 10; i++) {
        if (arr[i] < threshold) {
            count++;
        }
    }
    printf("Numbers less than %d = %d\n", threshold, count);
    return 0;
}
```

Sample Output

Numbers less than 20 = 6

Explanation — Step by Step (Simple Words)

1. The values below 20 are 5, 9, 3, 18, 7, 12 — that's 6 numbers.
2. This is Q12's exact program with '>' flipped to '<' — nothing else changes.
3. count starts at 0 and each thread privately increments its own copy whenever an element is below the threshold.
4. reduction(+:count) combines every thread's private count into the final total.

Note: Q12, Q18 and Q19 are the same counting skeleton three times — only the if-condition changes each time.

Q20. Find both the MAXIMUM and MINIMUM value of an array in the same loop.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>
int main() {
    int arr[8] = {12, 45, 3, 67, 21, 9, 88, 34};
    int max_val = arr[0], min_val = arr[0];
    #pragma omp parallel for reduction(max:max_val) reduction(min:min_val)
    for (int i = 0; i < 8; i++) {
        if (arr[i] > max_val) max_val = arr[i];
        if (arr[i] < min_val) min_val = arr[i];
    }
    printf("Maximum = %d, Minimum = %d\n", max_val, min_val);
    return 0;
}
```

Sample Output

Maximum = 88, Minimum = 3

Explanation — Step by Step (Simple Words)

1. You can write TWO reduction clauses on the same pragma line, one after another, when the operators are different (here max and min).
2. Both max_val and min_val start at arr[0] — a safe first guess for both.
3. Inside the loop each thread updates its own private best-max and best-min for its own chunk of the array, using two separate if-checks.
4. At the end OpenMP combines all threads' private max values into the overall maximum, and separately combines all the private min values into the overall minimum.

Note: This just merges Q7 and Q8 into a single loop — proof that a pragma line can carry more than one reduction clause at once.

Q21. Calculate the sum of squares of only the EVEN numbers in an array.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>
int main() {
    int i;
    long evenSquareSum = 0;
    int arr[10] = {1,2,3,4,5,6,7,8,9,10};
    #pragma omp parallel for reduction(+:evenSquareSum)
    for (i = 0; i < 10; i++) {
        if (arr[i] % 2 == 0) {
            evenSquareSum += arr[i] * arr[i];
        }
    }
    printf("Sum of squares of even numbers = %ld\n", evenSquareSum);
    return 0;
}
```

Sample Output

Sum of squares of even numbers = 220

Explanation — Step by Step (Simple Words)

1. The even numbers are 2,4,6,8,10. Their squares are 4,16,36,64,100, which add up to 220.
2. This combines THREE earlier tricks at once: sum-of-squares math (Q10), an if-filter (Q16), and the plain + reduction (Q4).
3. Each thread only squares-and-adds the even elements in its own chunk into its private evenSquareSum.
4. reduction(+:evenSquareSum) adds all the private partial sums together at the end.

Note: Shows how the teacher can 'stack' two twists (filter + power) on the same base skeleton — read the question carefully and apply both.

Q22. What is the difference between a PRIVATE and a SHARED variable? Show what goes wrong without private.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>
int main() {
    int id; // shared by default – NOT private
    #pragma omp parallel
    {
        id = omp_get_thread_num();
        printf("Thread says id = %d\n", id);
    }
}
```

```
}  
return 0;  
}
```

Sample Output

```
Thread says id = 2  
Thread says id = 2  
Thread says id = 3  
Thread says id = 3
```

Explanation — Step by Step (Simple Words)

1. By default, a variable declared BEFORE the parallel region (like `id` here) is SHARED — every thread reads and writes the exact same memory location.
2. Each thread sets `id = omp_get_thread_num()` and then immediately prints it, but because `id` is shared, one thread can overwrite `id` right before another thread's `printf` runs.
3. The result: the printed IDs can repeat or look wrong (e.g. two threads both printing '2') — this is a data race, and the output becomes unpredictable.
4. Adding `private(id)` — as in Q2 — fixes this instantly: each thread then gets its own separate copy of `id`, so no thread can ever overwrite another thread's value.
5. Rule of thumb: shared = one shared 'box' for everyone (risky for per-thread values); private = every thread gets its own 'box' (safe for per-thread values).

Note: Very common conceptual exam question. Answer in one line if asked short: 'shared variables are visible to and can be changed by all threads; private variables give each thread its own independent copy.'

Q23. Print which thread computed which loop iteration.

Answer (Code)

```
#include <stdio.h>  
#include <omp.h>  
int main() {  
    int i, id;  
    #pragma omp parallel for private(id)  
    for (i = 0; i < 8; i++) {  
        id = omp_get_thread_num();  
        printf("Iteration %d done by thread %d\n", i, id);  
    }  
    return 0;  
}
```

Sample Output

```
Iteration 0 done by thread 0  
Iteration 1 done by thread 0  
Iteration 2 done by thread 1  
Iteration 3 done by thread 1
```

```
Iteration 4 done by thread 2
...
```

Explanation — Step by Step (Simple Words)

1. This question combines THREE of the five directives at once: parallel for (splits the loop), private(id) (safe per-thread storage), and omp_get_thread_num() (which thread is running).
2. private(id) is needed here because every thread writes into id right before printing it — without private, threads would step on each other's id value, same risk as Q22.
3. OpenMP decides which chunk of iterations goes to which thread; it is common (but not guaranteed) for consecutive iterations to be handled by the same thread.
4. There is no reduction here because we are not combining a result — we're just observing/printing which thread did what, so no combining step is needed.

Note: If a question asks to 'show the work split between threads' rather than 'calculate a result', that's your signal: no reduction needed, just private + omp_get_thread_num().

Q24. Calculate the sum AND the count of array elements together, then find the average.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>
int main() {
    int i, sum = 0, count = 0;
    int arr[10] = {1,2,3,4,5,6,7,8,9,10};
    #pragma omp parallel for reduction(+:sum, count)
    for (i = 0; i < 10; i++) {
        sum += arr[i];
        count++;
    }
    float average = (float)sum / count;
    printf("Sum = %d, Count = %d, Average = %.2f\n", sum, count, average);
    return 0;
}
```

Sample Output

```
Sum = 55, Count = 10, Average = 5.50
```

Explanation — Step by Step (Simple Words)

1. Both sum and count use the SAME operator (+), so they can share one reduction clause: reduction(+:sum, count), separated by a comma — same trick as Q9's evenCount/oddCount.
2. Each thread privately builds up its own partial sum and its own partial count for its chunk of the array.
3. OpenMP adds all the private sums together into the final sum, and separately adds all the private counts together into the final count.
4. The average is computed afterward, outside the parallel region, exactly like Q13 — simple follow-up math doesn't need its own pragma.

Note: A slightly 'trickier' version of Q13: instead of assuming n is already known, the program also counts the elements itself using reduction.

Q25. Print the multiplication table of a number using OpenMP (parallel for, no reduction).

Answer (Code)

```
#include <stdio.h>
#include <omp.h>
int main() {
    int num = 5;
    #pragma omp parallel for
    for (int i = 1; i <= 10; i++) {
        printf("%d x %d = %d\n", num, i, num * i);
    }
    return 0;
}
```

Sample Output

```
5 x 1 = 5
5 x 4 = 20
5 x 2 = 10
5 x 5 = 25
5 x 3 = 15
...
(order may vary each run)
```

Explanation — Step by Step (Simple Words)

1. Each loop iteration (each row of the table) is completely independent of the others — thread A computing 5x3 never needs thread B's 5x7 result.
2. Because nothing needs to be combined at the end, there is NO reduction clause here — just a plain #pragma omp parallel for.
3. i is automatically private per iteration (same rule as Q3), so each thread safely computes its own num * i without clashing.
4. The rows print out of numeric order because different threads finish at different times — but every row still gets printed exactly once.

Note: Not every parallel-for question needs reduction! If nothing is being combined into one final answer, plain parallel for (like Q3) is enough.

Q26. Find the highest thread ID number used while printing from all threads.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>
int main() {
    int maxThread = 0;
    #pragma omp parallel reduction(max:maxThread)
    {
        int id = omp_get_thread_num();
        if (id > maxThread) maxThread = id;
        printf("Thread %d says hello\n", id);
    }
    printf("Highest thread ID used = %d\n", maxThread);
    return 0;
}
```

Sample Output

```
Thread 0 says hello
Thread 2 says hello
Thread 1 says hello
Thread 3 says hello
Highest thread ID used = 3
```

Explanation — Step by Step (Simple Words)

1. This uses plain `#pragma omp parallel` (not `parallel for`) — there's no loop to split, just a block that every thread runs once.
2. `id` is declared INSIDE the parallel block, so it is automatically local (private) to each thread — no explicit `private(id)` needed here.
3. `reduction(max:maxThread)` works on a parallel region the same way it works on `parallel for`: each thread keeps its own private 'best value so far', here its own thread ID.
4. After the region ends, OpenMP compares all threads' private `maxThread` values and keeps the largest — if there are 4 threads (IDs 0-3), the answer is 3.

Note: Proves reduction also works on plain `#pragma omp parallel`, not only on `parallel for` — the combining rule is the same either way.

Q27. Calculate the sum of the ABSOLUTE VALUES of array elements (array may contain negatives).

Answer (Code)

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
int main() {
    int i, sumAbs = 0;
    int arr[8] = {-5, 3, -8, 2, -1, 9, -4, 6};
    #pragma omp parallel for reduction(+:sumAbs)
    for (i = 0; i < 8; i++) {
        sumAbs += abs(arr[i]);
    }
}
```

```

}
printf("Sum of absolute values = %d\n", sumAbs);
return 0;
}

```

Sample Output

Sum of absolute values = 38

Explanation — Step by Step (Simple Words)

1. `abs(arr[i])` turns every negative number positive before adding, so -5 contributes 5, -8 contributes 8, and so on.
2. $5+3+8+2+1+9+4+6 = 38$, matching the printed result.
3. This is the exact same `reduction(+:sumAbs)` skeleton as Q4 — the only change is wrapping `arr[i]` in `abs()` before adding it.
4. Each thread privately totals the absolute values in its own chunk; OpenMP adds all the private totals together at the end.

Note: Whenever the loop body is a single expression added into the reduction variable (`abs()`, `squares`, `cubes...`), the pragma line itself never changes — only that one line inside the loop does.

Q28. Count how many elements are **POSITIVE** and how many are **NEGATIVE** in the same array.

Answer (Code)

```

#include <stdio.h>
#include <omp.h>
int main() {
    int i, posCount = 0, negCount = 0;
    int arr[8] = {-3, 5, -7, 2, 9, -1, 4, -6};
    #pragma omp parallel for reduction(+:posCount, negCount)
    for (i = 0; i < 8; i++) {
        if (arr[i] > 0)
            posCount++;
        else if (arr[i] < 0)
            negCount++;
    }
    printf("Positive count = %d\n", posCount);
    printf("Negative count = %d\n", negCount);
    return 0;
}

```

Sample Output

Positive count = 4
Negative count = 4

Explanation — Step by Step (Simple Words)

1. The positives are 5, 2, 9, 4 (4 of them); the negatives are -3, -7, -1, -6 (also 4) — matching the printed counts.
2. Exactly Q9's even/odd counting pattern reused: two counters reduced together with `reduction(+:posCount, negCount)`, separated by a comma.
3. Each thread privately tallies its own `posCount` and `negCount` for its chunk of the array using an `if / else if`.
4. OpenMP adds all threads' private `posCount` values together, and separately adds all the private `negCount` values together, at the end.

Note: If the array could also contain 0, add a plain `else` branch to count zeros too — same skeleton, one more private counter.

Q29. Why is reduction needed? Show a manual (unsafe) sum using `#pragma omp parallel` to explain.

Answer (Code)

```
#include <stdio.h>
#include <omp.h>
#define SIZE 8
int main() {
    int arr[SIZE] = {1,2,3,4,5,6,7,8};
    int sum = 0;    // shared – NOT private, NOT protected by reduction
    #pragma omp parallel
    {
        int id = omp_get_thread_num();
        int numThreads = 4;
        int chunk = SIZE / numThreads;
        int start = id * chunk;
        int end = start + chunk;
        for (int i = start; i < end; i++) {
            sum = sum + arr[i];    // multiple threads write to shared sum!
        }
    }
    printf("Sum (unsafe, may be wrong) = %d\n", sum);
    return 0;
}
```

Sample Output

Sum (unsafe, may be wrong) = 33 (or 36, or another wrong number — it changes between runs!)

Explanation — Step by Step (Simple Words)

1. Here `sum` is shared and updated directly by every thread with no reduction and no private — this is a manual, unprotected attempt to split the work by hand.
2. Each thread computes its own start/end range using `omp_get_thread_num()`, so the SPLITTING of work is fine — the PROBLEM is how the partial results get combined.
3. When two threads execute `sum = sum + arr[i]` at nearly the same instant, both may read the OLD value of `sum` before either writes back the new one — one thread's addition gets silently lost. This is the race condition mentioned back in Q4.

4. The correct fix is exactly what Q4 already does: add `reduction(+:sum)` to the `pragma` line. That automatically gives each thread a private, safe partial sum and combines them correctly at the end — no manual start/end splitting needed at all.

Note: This question exists to explain WHY reduction is used, not to be copied as a working answer — on the exam, always prefer `reduction(+:sum)` (Q4's version) over manual splitting like this.

Q30. What is the difference between `#pragma omp parallel` and `#pragma omp parallel for`?

Answer (Code)

```
// (A) #pragma omp parallel -- just creates threads
#pragma omp parallel
{
    printf("Hello\n"); // every thread runs this SAME line once
}

// (B) #pragma omp parallel for -- creates threads AND splits a loop
#pragma omp parallel for
for (int i = 0; i < 10; i++) {
    printf("%d\n", i); // loop iterations are SPLIT across threads
}
```

Sample Output

(A) prints "Hello" once PER THREAD (e.g. 4 times if there are 4 threads)
(B) prints each of `i = 0..9` exactly ONCE TOTAL, shared out across the threads

Explanation — Step by Step (Simple Words)

1. `#pragma omp parallel` only creates a team of threads and makes every thread run the ENTIRE block that follows — if there's a loop inside, every thread runs the WHOLE loop by itself, unless you add work-splitting logic yourself (like Q29).
2. `#pragma omp parallel for` does two jobs in one line: it creates the team of threads (like plain parallel) AND automatically divides a for-loop's iterations so each iteration runs on exactly ONE thread, not on all of them.
3. Use plain parallel when every thread should do the same full task (e.g. Q1, Q2, Q26). Use parallel for when you have one loop whose iterations should be shared out across threads (e.g. Q3 onward).
4. Quick memory trick: 'parallel' = copies of the whole block; 'parallel for' = slices of one loop.

Note: This wraps up the whole exam prep: every one of the 30 questions is either the plain-parallel skeleton (Q1, Q2, Q26, Q29) or the parallel-for(+reduction) skeleton (nearly everything else).

Quick Revision Cheat-Sheet

Directive	Purpose
<code>#pragma omp parallel</code>	Creates a team of threads that run the block together.
<code>#pragma omp parallel for</code>	Splits a for-loop's iterations across threads.
<code>private(var)</code>	Each thread gets its own private copy of a variable.
<code>reduction(+:sum)</code>	Safely combines a variable (e.g. sum) across all threads. Also works with *, max, min in place of +.
<code>omp_get_thread_num()</code>	Returns the calling thread's ID number.

Exam Strategy

Note: Almost every question in this guide reuses one of two skeletons: (1) plain parallel / parallel-for to split work with no combining (Q1, Q2, Q3, Q23, Q25, Q26), or (2) parallel-for + reduction to safely combine a result (nearly everything else). Spot which skeleton the question needs, slot in the new logic (factorial, even/odd, squares, cubes, threshold, positive/negative, absolute value...), and you're done. If it's a 'why' or 'difference between' question (Q22, Q29, Q30), explain the concept in words first, then back it up with a short code snippet.